



مطعم بيت المندي

وثيقة البنية المعمارية للنظام

RESTAURANT ORDER MANAGEMENT SYSTEM

1.0	الإصدار
BAM-ARCH-AR-20260616	رقم الوثيقة
١٦ يونيو ٢٠٢٦	تاريخ الإنشاء
توثيق معماري للنظام	نوع الوثيقة

تم التطوير بواسطة



Esnaad Tech

Technology Solutions & Software Development

حقوق التطوير والملكية التقنية

تم تطوير هذا النظام بواسطة:



Esnaad Tech

Technology Solutions & Software Development

جميع الحقوق التقنية والفكرية محفوظة.
هذه الوثيقة هي ملكية خاصة لشركة إسناد تك.
© Esnaad Tech 2026. جميع الحقوق محفوظة.

فهرس المحتويات

1	توثيق النظام الكامل - مطعم بيت المندي
2	المرحلة الأولى: فهرسة المشروع
3	المرحلة الثانية: شرح النظام
4	المرحلة الثالثة: تحليل الصفحات
5	المرحلة الرابعة: تحليل قاعدة البيانات
6	المرحلة الخامسة: دورة حياة الطلب
7	المرحلة السادسة: نظام العروض والباقات
8	المرحلة السابعة: نظام الفواتير
9	المرحلة الثامنة: نظام Tracking Token
10	المرحلة التاسعة: تحليل Stores
11	المرحلة العاشرة: تحليل Server Actions
12	المرحلة الحادية عشرة: تحليل الأمان
13	المرحلة الثانية عشرة: تحليل الأداء
14	المرحلة الثالثة عشرة: تحليل البنية المعمارية
15	المرحلة الرابعة عشرة: خريطة الاعتماديات
16	المرحلة الخامسة عشرة: التقرير النهائي

ملخص البنية المعمارية

نظرة عامة على التقنيات والخدمات المستخدمة في النظام

 اللغة TypeScript	 الإطار Next.js 14 (App Router)	 بنية النظام Hybrid: Layered + Feature-Based
 المصادقة Supabase Auth + RLS	 قاعدة البيانات Supabase (PostgreSQL) + Drizzle ORM	 التصميم TailwindCSS + CSS Variables
 الفواتير jsPDF + html2canvas	 الخرائط Leaflet + OSRM	 إدارة الحالة Zustand + localStorage
 التدقيق audit_logs	 المراقبة Sentry	 التحليلات PostHog

ملخص قاعدة البيانات

جداول النظام والعلاقات بينها — إجمالي 16 جدولاً

العلاقات	الحقول الرئيسية	الوصف	الجدول
orders (1:N)	id, full_name, phone	المستخدمون	users
order_status_history (1:N)	id, email, role, auth_user_id	المشرفون	admin_users
items (1:N)	id, name_ar, name_en, slug	التصنيفات	categories
item_prices (1:N), offer_items	id, category_id (FK), name_ar	الأطباق	items
items (N:1)	id, item_id (FK), sale_price	الأسعار	item_prices
offer_items (1:N), orders	id, title_ar, offer_type	العروض	offers
offers, items, item_prices	id, offer_id (FK), menu_item_id (FK)	مكونات العرض	offer_items
users, order_items (1:N)	id, order_number, status, total	الطلبات	orders
orders (N:1)	id, order_id (FK), item_name	عناصر الطلب	order_items
orders, admin_users	id, order_id (FK), new_status	سجل الحالة	order_status_history
—	id, image_url, show_on_homepage	المعرض	gallery_images
—	id, reviewer_name, rating, comment	التقييمات	reviews
—	id, setting_key (JSONB)	الإعدادات	site_settings
—	id, name_ar, address, phone	الفروع	branches
admin_users	id, action, details (JSON)	التدقيق	audit_logs
orders (1:1)	phone (PK), tracking_token	الرموز	customer_tokens

تدفقات النظام

تدفق إنشاء الطلب من التصفح حتى التسليم

1	التصفح	زيارة /menu، مشاهدة الأصناف والعروض، إضافة للسلة
2	السلة	تعبئة البيانات، اختيار الموقع، حساب رسوم التوصيل
3	الطلب	createOrder() → تحقق → حساب → إدراج في 4-6 جداول
4	الفاتورة	InvoiceModal → QR Code → PDF
5	التتبع	t/[token]/ ← رابط آمن مع حالة الطلب
6	الإدارة	admin/orders → تحديث الحالة مع optimistic locking
7	التقارير	تحليلات يومية/أسبوعية/شهرية مع تصدير

التقنيات الأساسية

رسوم التوصيل	حساب المسافة
base_fee + extra_km + weather + peak	Haversine + OSRM
رمز التتبع	رقم الطلب
UUID v4 ثابت لكل عميل	BAM-YYYYMMDD-XXXX
التحكم بالتزامن	حساب العروض
Optimistic Locking (version)	calculateOfferPrice() SSoT

توثيق النظام الكامل - مطعم بيت المندي

Bait Al Mandi Restaurant - Full System Architecture Documentation

المرحلة الأولى: فهرسة المشروع

هيكل المشروع (Project Tree)

```
baitalmandiwibapp/
├── package.json           # ملف تعريف المشروع والاعتماديات
├── next.config.js         # (الصور، التحسينات) إعدادات Next.js
├── tailwind.config.ts     # إعدادات TailwindCSS
├── tsconfig.json         # إعدادات TypeScript
├── .env.local             # (الخ، Supabase, Sentry) المتغيرات البيئية
├── drizzle.config.ts      # إعدادات Drizzle ORM
|
├── supabase/
|   └── migrations/        # ملفات الترحيل (12 ملفاً)
|       ├── 0000_small_ozymandias.sql  # الهيكل الأساسي: جميع الجداول
|       ├── 0001_remarkable_terrax.sql  # تعديلات التسعيرة
|       ├── 0002_fix_rls_policies.sql   # إصلاح سياسات RLS
|       ├── 0003_add_tracking_token.sql  # للأوردرات tracking_token إضافة
|       ├── 0004_add_offer_bundles.sql  # للباقات offer_items إضافة
|       ├── 0005_add_customer_tokens.sql # customer_tokens إضافة جدول
|       ├── 0006_add_order_offers.sql   # للعروض Snapshot إضافة جداول
|       ├── 0007_add_variant_to_offers.sql # offer_items دعم المتغيرات في
|       ├── 0008_add_delivery_system.sql # نظام التوصيل الجديد
|       ├── 0009_add_new_delivery_pricing.sql # تسعير التوصيل المحسن
|       ├── 0010_add_delivery_snapshot.sql # Snapshot بيانات التوصيل
|       └── 0011_add_show_on_homepage.sql # show_on_homepage إضافة حفل
|   └── meta/              # للترحيل Snapshot بيانات
|
└── src/
```



```
|   ├── middleware.ts           # Middleware لحماية مسارات الإدارة
|   ├── types.d.ts             # تعريفات الأنواع العامة
|   |
|   ├── app/                   # Next.js 14 App Router صفحات
|   |   ├── layout.tsx         # التخطيط الرئيسي (Navbar, Footer, Theme)
|   |   ├── page.tsx           # الصفحة الرئيسية (Hero, أراء, معرض, عروض)
|   |   ├── globals.css        # الأنماط العامة والمتغيرات
|   |   |
|   |   ├── menu/
|   |   |   └── page.tsx        # صفحة قائمة الطعام (أصناف + عروض)
|   |   |
|   |   ├── gallery/
|   |   |   └── page.tsx        # معرض الصور العام
|   |   |
|   |   ├── contact/
|   |   |   └── page.tsx        # صفحة الاتصال
|   |   |
|   |   ├── cart/
|   |   |   └── page.tsx        # صفحة السلة
|   |   |
|   |   ├── my-orders/
|   |   |   └── page.tsx        # صفحة إنشاء الطلب
|   |   |
|   |   ├── t/
|   |   |   └── [token]/
|   |   |       └── page.tsx    # Token صفحة تتبع الطلب بالـ
|   |   |
|   |   ├── track-order/
|   |   |   └── [orderId]/
```

```
| | | └─ page.tsx      # صفحة تتبع الطلب بال ID
| | |
| | └─ test-map/
| | | └─ page.tsx      # صفحة اختبار الخريطة
| | |
| | └─ admin/          # لوحة التحكم
| |   └─ layout.tsx    # تخطيط لوحة التحكم (Sidebar, Auth)
| |   └─ page.tsx      # الصفحة الرئيسية للوحة التحكم
| |   └─ login/
| |     └─ page.tsx    # صفحة تسجيل الدخول
| |     └─ orders/
| |       └─ page.tsx  # إدارة الطلبات
| |       └─ actions.ts # Server Actions للطلبات
| |       └─ menu/
| |         └─ page.tsx # إدارة الأطباق
| |       └─ categories/
| |         └─ page.tsx # إدارة التصنيفات
| |       └─ offers/
| |         └─ page.tsx # إدارة العروض والباقات
| |       └─ gallery/
| |         └─ page.tsx # إدارة معرض الصور
| |       └─ reviews/
| |         └─ page.tsx # إدارة التقييمات
| |       └─ reports/
| |         └─ page.tsx # التقارير والإحصائيات
| |       └─ delivery/
| |         └─ page.tsx # إعدادات التوصيل
| |       └─ settings/
| |         └─ page.tsx # إعدادات الموقع
```

```
| |
| |─ actions/                # Server Actions
| |   └─ orders.ts           # createOrder, getOrders, updateOrderStatus
| |     └─ orders-offers.ts  # getOrderOffers
| |
| |─ components/             # المكونات المشتركة
| |   └─ layout/
| |     └─ Navbar.tsx        # شريط التنقل
| |     └─ Footer.tsx        # التذييل
| |     └─ LoadingScreen.tsx # شاشة التحميل
| |   └─ ui/
| |     └─ Toast.tsx          # نظام الإشعارات (Toast)
| |   └─ invoice/
| |     └─ InvoiceModal.tsx    # نافذة الفاتورة المنبثقة
| |     └─ receipt-html.ts    # للفاتورة HTML توليد
| |   └─ admin/
| |     └─ reports/
| |       └─ OffersTab.tsx    # تقارير العروض
| |       └─ InvoicesTab.tsx # تقارير الفواتير
| |
| |─ store/                  # إدارة الحالة (Zustand)
| |   └─ cartStore.ts         # متجر السلة مع الحفظ المحلي
| |
| |─ lib/                    # الخدمات والأدوات المساعدة
| |   └─ supabase.ts          # (عام) Supabase عميل
| |   └─ offer-pricing.ts     # (SSoT) محرك حساب أسعار العروض
| |   └─ bundle-utils.ts      # أدوات الباقات والتوافق العكسي
| |   └─ delivery-pricing.ts  # (SSoT) محرك حساب رسوم التوصيل
| |   └─ delivery-routing.ts  # خدمات التوجيه والتوصيل
```

```
| | └─ location-validation.ts # التحقق من صحة الموقع
| | └─ whatsapp-message.ts    # بناء رسائل واتساب
| | └─ homepage-gallery.ts    # خدمة معرض الصفحة الرئيسية
| | └─ settings-context.tsx   # سياق إعدادات الموقع
| | └─ permissions.ts         # نظام الصلاحيات
| | └─ ordering.ts            # ترتيب العناصر
| | └─ printReport.ts         # طباعة التقارير
| | └─ validation.ts          # التحقق من صحة البيانات
| | └─ maps/
| |     └─ getDeliveryRoute.ts # توجيه OSRM
| |
| └─ db/                                # قاعدة البيانات
|     └─ schema.ts                 # تعريف جداول Drizzle ORM
|     └─ rls.sql                   # سياسات أمان الصفوف (RLS)
|
| └─ utils/
|     └─ supabase/                 # المساعدة Supabase أدوات
|         └─ client.ts             # عميل المتصفح
|         └─ server.ts             # عميل الخادم
|         └─ middleware.ts         # Middleware عميل
```

المرحلة الثانية: شرح النظام

ما هو هذا النظام؟

هو **نظام طلب طعام إلكتروني متكامل** لمطعم "بيت المندي" في صنعاء، اليمن. يسمح للعملاء بتصفح قائمة الطعام، مشاهدة العروض، إضافة الأصناف إلى السلة، وإنشاء طلبات توصيل مع إمكانية التتبع المباشر.

ما المشكلة التي يحلها؟

- 1. **أتمتة الطلبات:** تحويل الطلبات الهاتفية اليدوية إلى نظام إلكتروني منظم
- 2. **إدارة التوصيل:** حساب رسوم التوصيل تلقائياً بناءً على المسافة والظروف
- 3. **تتبع الطلبات:** تمكين العملاء من تتبع طلباتهم في الوقت الفعلي
- 4. **إدارة المطعم:** توفير لوحة تحكم شاملة للمديرين والموظفين
- 5. **التقارير:** إنشاء تقارير مالية وإدارية متكاملة

من هم المستخدمون؟

نوع المستخدم	الوصف	الصلاحيات
---	---	---
مطور (Developer)	مسؤول تقني كامل	جميع الصلاحيات دون استثناء
مدير (Manager)	مدير المطعم	إدارة الطلبات، الأصناف، التصنيفات، العروض، الصور، التقييمات، التقارير، الإعدادات
مدير طلبات (Order Manager)	موظف استقبال الطلبات	إدارة الطلبات فقط (عرض، تحديث حالة)
عميل (Customer)	زائر الموقع	تصفح القائمة، إنشاء الطلبات، تتبع الطلبات

المرحلة الثالثة: تحليل الصفحات

المسار	الصفحة	الغرض	الوصول	البيانات	الخدمات
---	---	---	---	---	---
/	الرئيسية	عرض العروض، القصة، التقييمات	عام	offers, reviews, gallery_images, site_settings	supabase, offer-pricing, homepage-gallery
menu/	قائمة الطعام	عرض الأصناف والعروض مع إضافة للسلة	عام	categories, items, item_prices, offers, offer_items	supabase, offer-pricing, cartStore
gallery/	معرض الصور	عرض صور المطعم مع تصفية وتكبير	عام	gallery_images	supabase
contact/	الاتصال	معلومات الفروع والتواصل	عام	site_settings, branches	supabase
cart/	السلة	عرض محتويات السلة وتعديلها	عام	cartStore (محلي)	cartStore
my-orders/	إنشاء الطلب	نموذج إنشاء طلب مع اختيار الموقع	عام	orders, site_settings, offers	supabase, orders.ts, cartStore
t/[token]/	تتبع الطلب	تتبع الطلب عبر الرابط الآمن	عام	orders, order_items, order_offers	supabase
track-/order/[orderId]	تتبع بال ID	تتبع الطلب برقم المعرف	عام	orders, order_offers	supabase, orders-offers
admin/	لوحة التحكم	ملخص وإحصائيات سريعة	مسؤول	orders	supabase
admin/orders/	الطلبات	إدارة وعرض الطلبات وتحديث الحالة	مسؤول	orders, order_items	supabase, orders.ts

المسار	الصفحة	الغرض	الوصول	البيانات	الخدمات
admin/menu/	الأطباق	إضافة وتعديل وحذف الأطباق	مسؤول	items, item_prices, categories	supabase
admin/categories/	التصنيفات	إدارة تصنيفات الأطباق	مسؤول	categories	supabase
admin/offers/	العروض	إنشاء وإدارة العروض والباقات	مسؤول	offers, offer_items, items, item_prices	supabase, offer-pricing
admin/gallery/	معرض الصور	إدارة صور المطعم	مسؤول	gallery_images	supabase
admin/reviews/	التقييمات	إدارة تقييمات العملاء	مسؤول	reviews	supabase
admin/reports/	التقارير	إحصائيات متقدمة للبيع	مسؤول	orders, order_offers	supabase
admin/delivery/	التوصيل	إعدادات التوصيل والمناطق	مسؤول	site_settings, branches	supabase
admin/settings/	الإعدادات	إعدادات الموقع العامة	مسؤول	site_settings	supabase

المرحلة الرابعة: تحليل قاعدة البيانات

مخطط ERD المنطقي

```

users (1) ——< orders (N) ——< order_items (N)

|
|
| ——< order_status_history (N)
|
|
| ——< order_offers (N) ——< order_offer_items (N)
|
|
| —— offers (N) ——< offer_items (N) ——< items (N)
|
|
|
| ——< item_prices (N)
|
|
| —— customer_tokens (1:1)

categories (1) ——< items (N)

admin_users (1) ——< order_status_history (N)

admin_users (1) ——< audit_logs (N)

```

شرح الجداول

1. users — العملاء

- **الغرض:** تخزين بيانات العملاء المسجلين

- **الحقول:** address, created_at, id, full_name, phone (فريد)

- **العلاقات:** مرتبط مع orders عبر customer_id

2. admin_users — المستخدمين الإداريون

- **الغرض:** تخزين بيانات المشرفين والموظفين

- **الحقول:** id, email, (فريد), auth_user_id, role (enum), full_name

- **الأدوار:** developer, manager, order_manager

- **ملاحظة:** مرتبط بحسابات Supabase Auth عبر auth_user_id

3. categories — تصنيفات الأطباق

- **الغرض:** تصنيف الأطباق (مثل: مندي، زربيان، مشروبات)

- **الحقول:** slug, name_en, name_ar, id (فريد), is_active, sort_order, image, icon

- **العلاقات:** مرتبط مع items عبر category_id

4. items — الأطباق

- **الغرض:** تخزين بيانات الأطباق والوجبات

- **الحقول:** id, category_id (FK), name_ar, name_en, description_ar, description_en, image,

is_active, sort_order, is_available, is_best_seller

- **العلاقات:**

- belongs_to: categories (category_id)

- has_many: item_prices (حجمات وأسعار مختلفة)

- has_many: offer_items (كمكونات في الباقات)

5. item_prices — أسعار الأطباق حسب الحجم

- **الغرض:** تخزين الأسعار المختلفة لنفس الطبق (صغير/وسط/كبير)

- **الحقول:** id, item_id (FK), size_label_ar, size_label_en, serves, original_price, sale_price,

is_active

- **العلاقات:** belongs_to: items

6. offers — العروض والباقات

- **الغرض:** تخزين العروض الترويجية والباقات المجمعة

- **الحقول:** id, item_id (FK قديم), title_ar, title_en, description_ar, description_en, discount_percent, sale_price, discount_amount, offer_type, is_active, start_date, end_date, image, status, deleted_at

- **أنواع العروض:** fixed_price, percentage_discount, amount_discount, free_item

- **العلاقات:**

- belongs_to: items (قديم - للعروض الفردية)

- has_many: offer_items (للجداول)

7. offer_items — مكونات الباقة

- **الغرض:** تخزين المنتجات المكونة للباقة مع الكميات والأسعار

- **الحقول:** id, offer_id (FK), menu_item_id (FK), quantity, price_id (FK), variant_name, unit_price

- **العلاقات:**

- belongs_to: offers

- belongs_to: items (عبر menu_item_id)

- belongs_to: item_prices (عبر price_id لتحديد الحجم)

8. cart_sessions — جلسات السلة

- **الغرض:** تخزين جلسات السلة للزوار غير المسجلين

- **الحقول:** id, session_id, phone, expires_at

9. cart_items — عناصر السلة

- **الغرض:** تخزين العناصر في السلة لكل جلسة

- **الحقول:** id, session_id (FK), item_id (FK), price_id (FK), quantity, unit_price

10. orders — الطلبات (الجدول الرئيسي)

- **الغرض:** تخزين جميع الطلبات مع بياناتها الكاملة

- **الحقول:** +30 حقلاً تشمل:

- المعرفات: id, order_number (فريد), tracking_token

- العميل: customer_id, customer_name, customer_phone

- التوصيل: delivery_address, delivery_latitude, delivery_longitude, delivery_zone,

delivery_distance_km, delivery_duration_minutes

- المالية: subtotal, delivery_fee, tax_amount, total_amount

- حالة الطلب: status, version (للتحكم في التزامن)

- رسوم التوصيل المفصلة: base_delivery_fee_amount, extra_distance_km,

extra_fee_amount, weather_fee_amount, peak_fee_amount, peak_percentage_used

- الأرشفة: is_archived, archived_at, is_deleted, deleted_at

- **الحالة (Status Flow):** pending → confirmed → preparing → on_the_way → delivered |

cancelled

- **العلاقات:**

belongs_to: users (customer_id) -

has_many: order_items -

has_many: order_status_history -

has_one: order_offers -

11. order_items — عناصر الطلب (Snapshot)

- **الغرض:** تخزين نسخة من الأصناف عند إنشاء الطلب (حتى لو تغير السعر لاحقاً)

- **الحقول:** id, order_id (FK), category_name, item_name, size_label, quantity, unit_price, subtotal, total_price

12. order_status_history — سجل حالة الطلب

- **الغرض:** تتبع جميع التغييرات في حالة الطلب مع المشرف المسؤول

- **الحقول:** id, order_id (FK), old_status, new_status, changed_by_admin_id (FK), created_at

13. gallery_images — صور المعرض

- **الغرض:** تخزين صور المطعم للمعرض والصفحة الرئيسية

- **الحقول:** id, image_url, caption_ar, caption_en, category, sort_order, is_active, show_on_homepage (جديد)

14. reviews — تقييمات العملاء

- **الغرض:** تخزين تقييمات ومراجعات العملاء

- **الحقول:** id, reviewer_name, reviewer_image, rating, comment_ar, comment_en, source, is_featured

15. site_settings — إعدادات الموقع

- **الغرض:** تخزين جميع إعدادات الموقع (SSoT للإعدادات)

- **الحقول:** id, setting_key (فريد), updated_at, value (JSONB)

- **الأمثلة:** restaurant_name, currency, phone_reservations, delivery_enabled, base_delivery_fee, max_delivery_distance_km, min_order_amount

16. branches — الفروع

- **الغرض:** تخزين بيانات فروع المطعم

- **الحقول:** id, name_ar, name_en, address, phone, map_url, working_hours, is_active

17. audit_logs — سجل التدقيق

- **الغرض:** تسجيل جميع العمليات الهامة على النظام

- **الحقول:** id, entity_id, entity_type, action (enum), details, admin_id (FK), created_at

18. customer_tokens — رموز العملاء

- **الغرض:** ربط رقم هاتف العميل بـ tracking_token ثابت للخصوصية والأمان

- **الحقول:** phone (PK), tracking_token (default gen_random_uuid()), created_at

19. order_offers — العروض في الطلب (Snapshot)

- **الغرض:** تخزين نسخة من بيانات العرض وقت الطلب

- **الحقول:** id, order_id (FK), offer_id, offer_name, offer_type, original_price, discount_amount, discount_percent, final_price

20. order_offer_items — عناصر العرض في الطلب (Snapshot)

- **الغرض:** تخزين نسخة من مكونات الباقية وقت الطلب

- **الحقول:** id, order_offer_id (FK), item_id, item_name, size_label, quantity, unit_price, total_price

21. scheduled_reports — التقارير المجدولة

- **الغرض:** جدولة إرسال التقارير الدورية عبر البريد الإلكتروني

- **الحقول:** id, report_type, period (enum), recipients (array), is_active, last_sent_at

22. order_sequences — تسلسل أرقام الطلبات

- **الغرض:** توليد أرقام طلبات متسلسلة يومية

- الحقول: last_number, id (فريد), sequence_date

المرحلة الخامسة: دورة حياة الطلب

تدفق الطلب الكامل

1. تصفح القائمة

- |
- /menu العميل يزور
- يشاهد الأصناف والعروض
- (cartStore) يضيف الأصناف إلى السلة

2. السلة

- |
- /my-orders تظهر السلة في
- يعنى العميل بياناته (الاسم، الهاتف، العنوان)
- يختار الموقع على الخريطة (اختياري)
- تظهر رسوم التوصيل تلقائياً
- يختار طريقة الدفع (نقدي/تحويل/محفظة)

3. إنشاء الطلب

- |
- "العميل يضغط" إتمام الطلب
- request → createOrder() Server Action
- |
- التحقق (Validation):
 - | — اسم العميل (مطلوب)
 - | — رقم الهاتف (صيغة يمنية صحيحة)
 - | — العنوان (10 أحرف على الأقل)
 - | — الأصناف (مطلوب عنصر واحد على الأقل)
 - | — المسافة (ضمن نطاق التوصيل)

```
|
|
| — حساب رسوم التوصيل:
|
|   | — Haversine للمسافة المستقيمة
|
|   | — OSRM للمسافة الفعلية
|
|   | — base_fee + extra_km_fee
|
|   | — weather_fee (إن وجد)
|
|   | — peak_hours_fee (إن وجد)
|
|   | — max_delivery_distance_km التحقق من
|
|
| — إنشاء رقم الطلب:
|
|   | — الصيغة: BAM-YYYYMMDD-XXXX
|
|   | — optimistic locking مع order_sequences استخدام
|
|   | — fallback عشوائي في حال فشل التوليد
|
|
| — إدارة رمز التتبع:
|
|   | — موجود للهاتف tracking_token البحث عن
|
|   | — جديد إذا لم يوجد tracking_token إنشاء
|
|
| — حساب العرض (إن وجد):
|
|   | — offer_items جلب العرض مع
|
|   | — التحقق من صلاحية العرض
|
|   | — calculateOfferPrice() حساب السعر عبر
|
|   | — إضافة السعر المخفض إلى إجمالي الطلب
|
|
| — التحقق من الحد الأدنى:
|
|   | — total ≥ min_order_amount
|
|   | — رفض الطلب إذا كان أقل
|
|
| — إنشاء الطلب في قاعدة البيانات:
```


- | — orders إدراج في
- | — order_items إدراج في
- | — order_status_history إدراج في
- | — (إن وجد عرض) order_offers + order_offer_items إدراج في
- |
- | — معالجة الدفع:
- | — cash: لا إجراء إضافي
- | — transfer: فتح واتساب مع رسالة التحويل
- | — wallet: فتح واتساب مع رسالة المحفظة
- |
- | — response → order + tracking_token

4. الفاتورة

- |
- | — InvoiceModal: عرض الفاتورة PDF
- | — receipt-html: توليد HTML للفاتورة
- | — QR Code: رمز QR للتنبع
- | — html2canvas + jsPDF: تحويل HTML إلى PDF

5. التنبع

- |
- | — /t/[token] ← رابط آمن للتنبع
- | — يعرض حالة الطلب وموقعه
- | — يعرض تفاصيل الفاتورة
- | — يعرض خريطة التوصيل (اختياري)

6. إدارة الطلب (لوحة التحكم)

- |
- | — admin/orders: عرض جميع الطلبات

— تحديث الحالة: pending → confirmed → preparing → on_the_way → delivered

— التحكم في التزامن: optimistic locking (version)

— تسجيل كل تغيير في order_status_history

— إلغاء الطلب مع تسجيل الإلغاء

7. التقارير

—

— تقارير يومية/أسبوعية/شهرية

— إجمالي المبيعات

— إحصائيات العروض

— تحليلات التوصيل

— Excel تصدير

المرحلة السادسة: نظام العروض والباقات

كيف يعمل نظام العروض؟

نظام العروض مبني على مفهوم **SSoT (Single Source of Truth)** حيث أن `calculateOfferPrice()` في الملف `src/lib/offer-pricing.ts` هي المصدر الوحيد لحساب أسعار العروض في جميع أنحاء النظام.

أنواع العروض

النوع	الوصف	مثال
---	---	---
fixed_price	سعر ثابت للباقة بغض النظر عن مكوناتها	باقة بـ 1500 ريال
percentage_discount	خصم نسبة مئوية من إجمالي مكونات الباقة	خصم 20%
amount_discount	خصم مبلغ ثابت من إجمالي الباقة	خصم 500 ريال
free_item	أرخص منتج في الباقة مجاني	اشتر 3 وادفع 2

تخزين الباقيات

عند إنشاء أو تعديل عرض (عبر `admin/offers/`):

1. **اختيار المنتجات:** يختار المدير المنتجات المكونة للباقة مع الكميات

2. **تحديد الحجم:** يختار الحجم/المتغير لكل منتج (اختياري)

3. **حساب `unit_price`:** يتم حساب السعر الفعال لكل منتج باستخدام `resolveItemPrice()` الذي يفضل `sale_price` على `original_price`

4. **تخزين السعر:** يتم تخزين `unit_price` في جدول `offer_items` كنسخة احتياطية (snapshot)

5. **حساب السعر النهائي:** يتم حساب السعر النهائي للعرض عبر `calculateOfferPrice()`

خوارزمية calculateOfferPrice

```
function calculateOfferPrice(input):  
  
    originalPrice = sum(product.unitPrice * product.quantity)  
  
    switch input.offerType:  
  
        fixed_price:      finalPrice = salePrice  
  
        percentage_discount: finalPrice = originalPrice * (1 - discountPercent/100)  
  
        amount_discount:   finalPrice = max(0, originalPrice - discountAmount)  
  
        free_item:         finalPrice = originalPrice - cheapestItemPrice  
  
  
    discountAmount = originalPrice - finalPrice  
  
    discountPercent = round(discountAmount / originalPrice * 100)  
  
    return { originalPrice, finalPrice, discountAmount, discountPercent }
```

هل calculateOfferPrice هي المصدر الوحيد؟

نعم. جميع أنحاء النظام تستخدم نفس الدالة:

- menu/page.tsx/ - عرض العروض في قائمة الطعام
- admin/offers/page.tsx/ - معاينة السعر عند إنشاء العرض
- src/actions/orders.ts - حساب سعر العرض عند إنشاء الطلب
- src/app/page.tsx - عرض العروض في الشريط المتحرك (تم إصلاحه)

المرحلة السابعة: نظام الفواتير

الملفات المسؤولة

- مكون نافذة الفاتورة `src/components/invoice/InvoiceModal.tsx`
- توليد HTML للفاتورة `src/components/invoice/receipt-html.ts`

كيفية إنشاء الفاتورة

1. يتم فتح `InvoiceModal` بعد إنشاء الطلب أو من صفحة التتبع
2. يتم تمرير بيانات الطلب (order, items, offer info) إلى المكون
3. يتم عرض الفاتورة بتنسيق HTML منسق بالكامل
4. يتم إنشاء رمز QR للتتبع (باستخدام مكتبة `qrcode.react`)
5. يمكن تحويل الفاتورة إلى PDF عبر `jsPDF` + `html2canvas`
6. يمكن طباعة الفاتورة مباشرة

مكونات الفاتورة

- **رقم الطلب:** BAM-YYYYMMDD-XXXX
- **بيانات العميل:** الاسم، الهاتف، العنوان
- **الأصناف:** اسم الصنف، الحجم، الكمية، السعر
- **العروض:** اسم العرض، السعر الأصلي، الخصم، السعر النهائي
- **الإجماليات:** subtotal, delivery_fee, total_amount
- **طريقة الدفع:** cash, transfer, wallet
- **رمز QR:** رابط التتبع الآمن

المرحلة الثامنة: نظام Tracking Token

ما هو tracking_token؟

هو معرف فريد آمن (UUID v4) يُستخدم لتتبع الطلبات دون الحاجة إلى كشف رقم الطلب أو بيانات العميل الحساسة.

كيف يتم إنشاؤه؟

1. عند إنشاء طلب جديد، يبحث النظام عن `customer_tokens` برقم هاتف العميل
2. إذا وُجد رمز سابق، يتم إعادة استخدامه (رمز ثابت لكل عميل)
3. إذا لم يوجد، يتم إنشاء رمز جديد عبر `crypto.randomUUID()`

المزايا الأمنية

- **الخصوصية:** لا يمكن تخمين رمز التتبع (UUID عشوائي)
- **الثبات:** نفس العميل له نفس الرمز دائماً (حتى مع طلبات متعددة)
- **الربط الآمن:** الرمز مرتبط برقم الهاتف فقط، وليس بأي معلومات حساسة أخرى
- **المسار الآمن:** `t/[token]/` يتطلب الرمز الصحيح للوصول

المخاطر المحتملة

- إذا تسرب `tracking_token` ، يمكن لأي شخص تتبع طلبات ذلك العميل
- الحل المقترح: إضافة طبقة تحقق إضافية (مثل رمز SMS) للوصول إلى صفحة التتبع

المرحلة التاسعة: تحليل Stores

cartStore (Zustand + Persist)

الملف: `src/store/cartStore.ts`

الحالة (State):

```
interface CartState {  
  
  items: CartItem[] // مصفوفة عناصر السلة  
  
}
```

الإجراءات (Actions):

الوصف	الإجراء
---	---
إضافة عنصر (يزيد الكمية إذا موجود)	<code>addToCart(item)</code>
حذف عنصر بالكامل	<code>removeFromCart(id)</code>
تحديث الكمية (يحذف إذا 0)	<code>updateQuantity(id, qty)</code>
تفريغ السلة بالكامل	<code>()clearCart</code>
حساب إجمالي السعر	<code>()getCartTotal</code>
حساب عدد العناصر	<code>()getTotalItems</code>

الحفظ المحلي: يتم حفظ السلة في `localStorage` تحت المفتاح `bam-cart-storage` عبر `zustand/middleware/persist`

هيكل CartItem:

```
interface CartItem {  
  
  id: string;           // معرف العنصر  
  
  name: string;         // اسم العنصر  
  
  price: number;        // السعر  
  
  quantity: number;     // الكمية  
  
  size?: string;        // الحجم  
  
  category?: string;    // التصنيف  
  
  image?: string;       // الصورة  
  
  isOffer?: boolean;    // هل هو عرض؟  
  
  offerId?: string;     // معرف العرض  
  
  offerType?: string;   // نوع العرض  
  
  originalPrice?: number; // السعر الأصلي  
  
  discountAmount?: number; // قيمة الخصم  
  
  discountPercent?: number; // نسبة الخصم  
  
  bundleItems?: BundleSubItem[]; // مكونات الباقة  
  
}
```


المرحلة العاشرة: تحليل Server Actions

جدول Server Actions

الاسم	الوظيفة	المدخلات	المخرجات	الجدول المتأثرة
---	---	---	---	---
createOrder	إنشاء طلب جديد	CreateOrderInput	order object	orders, order_items, order_status_history, order_offers, order_offer_items, customer_tokens, order_sequences
getOrders	جلب قائمة الطلبات	filters (status, is_archived)	[]order	orders, order_items
getOrderById	جلب طلب بالـ ID	orderId	order	orders, order_items
updateOrderStatus	تحديث حالة الطلب	orderId, newStatus, adminId	order	orders, order_status_history
calculateDeliveryFeeServer	حساب رسوم التوصيل	lat, lng	{ ... , fee, distanceKm }	site_settings, branches
getOrderOffers	جلب عروض الطلب	orderId	[]OrderOfferSnapshot	order_offers, order_offer_items

الـ Critical Actions

1. **createOrder** — الأكثر أهمية. يحتوي على منطق معقد:

- التحقق من جميع المدخلات

- حساب المسافة والرسوم
- إنشاء رقم الطلب (مع optimistic locking)
- إدارة tracking_token
- حساب العروض
- إنشاء السجلات في 4-6 جداول مختلفة
- 2. **updateOrderStatus** — التحكم في حالة الطلب مع:
- optimistic locking (باستخدام حقول version)
- تسجيل كل تغيير في order_status_history

المرحلة الحادية عشرة: تحليل الأمان

المصادقة (Authentication)

- **Supabase Auth**: يستخدم Supabase للمصادقة
- **Admin Login**: /admin/login يتصل بـ Supabase Auth
- **Middleware**: src/middleware.ts يحمي جميع مسارات */admin/
- **Session**: يتم التحقق من الجلسة عبر supabase/ssr@

الصلاحيات (Authorization)

- **نظام الأدوار**: developer, manager, order_manager
- **التحقق في العميل**: src/lib/permissions.ts يحدد الصلاحيات لكل مسار
- **التحقق في الخادم**: يتم التحقق من الدور في Server Actions

حماية التتبع

- **Tracking Token:** UUID عشوائي يصعب تخمينه
- **ربط الهاتف:** الرمز مرتبط برقم الهاتف فقط

المخاطر الأمنية الحالية

المخاطرة	الوصف	الخطورة
---	---	---
عدم وجود Rate Limiting	يمكن إرسال طلبات غير محدودة	متوسطة
تسريب tracking_token	يمكن تتبع طلبات العميل إذا تسرب الرابط	منخفضة
عدم التحقق من CSRF	Server Actions قد تكون عرضة لـ CSRF	منخفضة
Supabase RLS فقط	بعض الاستعلامات تعتمد فقط على RLS	متوسطة
كلمة سر Supabase في ENV	كلمة سر قاعدة البيانات في env.local	عالية

المرحلة الثانية عشرة: تحليل الأداء

الاستعلامات الثقيلة

1. **صفحة الطلبات (/admin/orders):** تجلب جميع الطلبات مع العناصر في استعلام واحد
2. **صفحة التقارير (/admin/reports):** تحليلات معقدة على Data Warehouse
3. **إنشاء الطلب:** 5-7 استعلامات متتالية في createOrder

مشاكل N+1 المحتملة

1. **صفحة /menu:** تجلب الأصناف مع الأسعار (حل جيد باستخدام select مدمج)
2. **صفحة /admin/offers:** تجلب العروض مع المنتجات (select مدمج)

3. **createOrder**: استعلام منفصل للعرض ثم للعناصر

توصيات الأداء حسب الأولوية

1. **عالي**: إضافة Indexes على `orders.created_at` و `order_sequences.sequence_date`

2. **عالي**: إضافة Pagination على `admin/orders/`

3. **متوسط**: استخدام `React.Suspense` مع التحميل المسبق للبيانات

4. **متوسط**: تحسين استعلامات التقارير باستخدام `Materialized Views`

5. **منخفض**: إضافة `Redis/Upstash` للتخزين المؤقت لإعدادات الموقع

المرحلة الثالثة عشرة: تحليل البنية المعمارية

البنية الحالية

النظام يستخدم **Hybrid Architecture** تجمع بين:

1. **Layered Architecture** (طبقات):

- طبقة العرض: المكونات (Components)

- طبقة المنطق: الخدمات والأدوات (Lib)

- طبقة البيانات: قاعدة البيانات (DB)

- طبقة API: Server Actions + API Routes

2. **Feature-Based** (حسب الميزات):

- كل ميزة في ملف منفصل

- الصفحات مقسمة حسب الوظيفة

نقاط القوة

- استخدام Server Actions لتقليل API Routes
- تطبيق SSoT في حساب الأسعار والرسوم
- Snapshot للبيانات المهمة (order_items, order_offers)
- Optimistic locking في تحديث الطلبات

نقاط الضعف

- عدم وجود طبقة Service منفصلة (الكود في الصفحات مباشرة)
- تكرار منطق جلب البيانات بين الصفحات
- عدم وجود Validation Layer موحدة
- خلط بين Client و Server logic في بعض الملفات

البنية المقترحة

```
src/

├─ app/                # Pages (Presentation Layer)
├─ components/         # Shared Components
├─ features/           # Feature Modules
│   └─ orders/
│       ├── actions.ts  # Server Actions
│       ├── service.ts  # Business Logic
│       ├── types.ts    # Types
│       └─ components/  # Feature Components
│   └─ offers/
│       └─ delivery/
├─ lib/                # Core Services
├─ db/                 # Database
└─ store/              # State Management
```

المرحلة الرابعة عشرة: خريطة الاعتماديات

الصفحة الرئيسية (/)

```
|
|
└─ supabase (صور المعرض، العروض، التقييمات)
    └─ gallery_images, offers, reviews tables
    |
    |
└─ calculateOfferPrice (أسعار العروض)
    └─ offer-pricing.ts
    |
    |
└─ fetchHomepageGalleryImages (معرض الصور)
    └─ homepage-gallery.ts
    |
    |
└─ useSettings (إعدادات الموقع)
    └─ settings-context.tsx
    |
    |
└─ cartStore (حالة السلة)
    └─ cartStore.ts
```

صفحة قائمة الطعام (/menu)

```
|
|
└─ supabase (الأصناف، التصنيفات، العروض)
    └─ items, categories, offers, offer_items tables
    |
    |
└─ calculateOfferPrice + resolveItemPrice
    └─ offer-pricing.ts
    |
    |
└─ useSettings
```

```
|   └─ settings-context.tsx
```

```
|
```

```
└─ cartStore
```

```
    └─ cartStore.ts
```

```
إنشاء الطلب (createOrder)
```

```
|
```

```
└─ Server Actions
```

```
|   └─ orders.ts
```

```
|       └─ fetchDeliverySettings (site_settings + branches)
```

```
|       └─ calculateRoute (OSRM)
```

```
|       └─ calculateDeliveryFee (delivery-pricing.ts)
```

```
|       └─ calculateOfferPrice (offer-pricing.ts)
```

```
|       └─ generateOrderNumber (order_sequences)
```

```
|       └─ manageTrackingToken (customer_tokens)
```

```
|       └─ createOrderInserts (orders + order_items + order_status_history + order_offers)
```

المرحلة الخامسة عشرة: التقرير النهائي

1. Executive Summary

بيت المندي هو نظام طلب طعام إلكتروني متكامل مبني على **Next.js 14** مع **Supabase** كقاعدة بيانات. يقدم النظام حلاً كاملاً لإدارة مطعم يمّني يشمل: قائمة طعام رقمية، عروض وباقات، طلبات توصيل مع تتبع مباشر، فواتير إلكترونية، ولوحة تحكم شاملة.

2. Architecture Overview

Framework: Next.js 14 (App Router) -

Language: TypeScript -

Styling: TailwindCSS + CSS Variables (Dark/Light mode) -

Database: Supabase (PostgreSQL) + Drizzle ORM -

State: Zustand + localStorage persist -

Auth: Supabase Auth + RLS -

Maps: Leaflet + OSRM routing -

PDF: jsPDF + html2canvas -

Analytics: PostHog -

Monitoring: Sentry -

3. Database Overview

- **22 جدولاً** في قاعدة البيانات (بما فيها الأنواع المعددة Enums)

- **المهاجرون (Migrations):** 12 ملف ترحيل

- **نظام Snapshot للبيانات التاريخية:** order_items, order_offers, order_offer_items

- **نظام RLS:** سياسات أمان صفوف للجداول العامة والإدارية

4. Technical Debt

1. تكرار منطق calculateOfferPrice بين العميل والخادم

2. استخدام any في العديد من الأماكن بدلاً من TypeScript strict

3. عدم وجود Validation Layer مركزية

4. مسار تخزين Supabase به خطأ إملائي: "gellary" بدلاً من "gallery"

5. عدم وجود اختبارات أتمتة (Unit Tests)

6. بعض الملفات تجمع بين منطق Client و Server دون فصل واضح

5. المخاطر (Risks)

المخاطرة	التأثير	الاحتمالية	الإجراء المقترح
---	---	---	---
عدم وجود Pagination في الطلبات	بطء في الصفحة مع زيادة الطلبات	عالية	إضافة Pagination فوراً
عدم وجود Rate Limiting	إمكانية إغراق الخادم بالطلبات	متوسطة	إضافة Rate Limiting
تكرار كود حساب العروض	تناقض في الأسعار مع التعديلات	منخفضة (تم حلها)	مراجعة دورية
عدم وجود اختبارات	صعوبة اكتشاف الأخطاء	عالية	إضافة اختبارات تدريجياً

6. التوصيات (Recommendations)

1. **فوري:** إضافة Pagination إلى صفحة الطلبات

2. **فوري:** إضافة Indexes على الحقول المستخدمة في الفرز (created_at, order_number)

3. **قصير المدى:** إنشاء طبقة Service منفصلة للمنطق التجاري

4. **قصير المدى:** إضافة Rate Limiting لـ Server Actions

5. **متوسط المدى:** كتابة اختبارات وحدة للخدمات الأساسية

6. **متوسط المدى:** فصل الكود إلى وحدات (Feature Modules)

7. **طويل المدى:** إضافة CI/CD pipeline

8. **طويل المدى:** تحسين LCP بتحميل الصور عبر Next/Image

7. Future Roadmap

1. **Q1:** إضافة دفع إلكتروني عبر بوابات الدفع اليمينية

2. Q1: نظام الإشعارات (SMS/WhatsApp) للعملاء

3. Q2: تطبيق جوال (React Native)

4. Q2: برنامج ولاء العملاء ونظام النقاط

5. Q3: التكامل مع منصات التوصيل الخارجية

6. Q3: نظام إدارة المخزون

7. Q4: تقارير متقدمة مع AI وتحليلات تنبؤية

تم إنشاء هذه الوثيقة في 16 يونيو 2026

جميع الحقوق محفوظة © 2026 مطعم بيت المندي



شكراً لقراءة الوثيقة

BAIT AL MANDI — نظام إدارة الطلبات

تم التطوير بواسطة



Esnaad Tech

Technology Solutions & Software Development

١٦ يونيو ٢٠٢٦ | © 2026 جميع الحقوق محفوظة